컴퓨터 구조

**[개인과제-2] 임베디드 시스템 개발용 보드 조사**

하드웨어 구성과 특징 및 컴퓨터 구조 관점의 비교

Arduino UNO R3 / ESP32-DevKitC V4
Raspberry Pi 5 / BeagleBone Black


전공 컴퓨터공학

학번 2023150038

학년 4학년

성명 강다빈


자료 확인일 2026년 9월 24일

## 01 조사 목적과 컴퓨터 구조의 공통 틀

임베디드 시스템은 기계나 전자 장치 안에서 특정 기능을 수행하는 컴퓨터이다. 개발보드는 프로세서에 전원 회로와 입출력 연결부 등을 더해, 응용 프로그램과 주변장치를 시험할 수 있도록 만든 하드웨어이다. 같은 개발보드라도 센서 제어기, 통신 장치, 영상 처리 장치 중 어떤 역할을 맡느냐에 따라 필요한 구조가 달라진다. [1]

### 조사 범위와 선정 기준

본 보고서는 단순 제어용 MCU 보드인 Arduino UNO R3, 무선 통신을 통합한 ESP32-DevKitC V4, 고성능 Linux 단일보드 컴퓨터인 Raspberry Pi 5, 실시간 입출력 보조 프로세서를 갖춘 BeagleBone Black을 조사한다. MCU는 CPU와 메모리·주변장치를 한 칩에 통합한 마이크로컨트롤러이며, SBC는 컴퓨터의 주요 기능을 한 보드에 구성한 단일보드 컴퓨터를 뜻한다.

모델 혼동을 막기 위해 ESP32는 ESP32-WROOM-32E-N4 모듈 장착 구성, Raspberry Pi 5는 4GB 모델, BeagleBone Black은 Rev C를 기준으로 삼았다. 공식 문서의 사양을 조사했으며 실물 측정이나 성능 벤치마크를 수행한 보고서는 아니다. 활용 사례와 선택 의견은 사양에 근거한 설계 관점의 분석이다.

| 강의 1장의 구성 요소 | 개발보드에서 확인할 항목 |
|---|---|
| CPU | 명령어 실행, 데이터 처리, 비트 폭, 코어 수, 클럭 |
| 기억장치 | 실행 중 데이터를 두는 RAM과 전원이 꺼져도 남는 저장 공간 |
| 입출력장치 | 센서 입력, LED·모터 출력, 통신·디스플레이 연결 |
| 시스템 버스 | CPU·메모리·제어기 사이의 데이터와 제어 신호 전달 |
| 시스템 소프트웨어 | 부트로더, 펌웨어, RTOS 또는 Linux의 역할 |

### 보고서 구성

3~4쪽 Arduino UNO R3, 5~6쪽 ESP32-DevKitC V4, 7~8쪽 Raspberry Pi 5, 9~10쪽 BeagleBone Black을 다룬다. 11쪽에서 보드별 차이와 선정 기준을 종합하고, 12쪽에 참고문헌, 마지막 13쪽에 개인 소감을 제시한다.

개념 및 강의 연계: [1] 제1장 컴퓨터시스템 개요, PDF 6~8·11~13·56·62쪽.

## 02 Arduino UNO R3의 하드웨어 구성

Arduino UNO R3는 ATmega328P를 중심으로 구성된 MCU 개발보드이다. 주 응용 프로그램을 실행하는 칩과 USB 통신을 처리하는 칩이 구분되며, 운영체제 없이 작은 제어 프로그램을 반복 실행하는 구성을 이해하기에 적합하다. [2][3]




**그림 1. Arduino UNO R3 보드. 출처: Arduino 공식 데이터시트 [3], 1쪽.**

| 구성 항목 | 사양과 역할 |
|---|---|
| 프로세서 | ATmega328P, 8비트 AVR, 16MHz |
| 메모리 | 플래시 32KB, SRAM 2KB, EEPROM 1KB |
| 디지털 입출력 | 14핀, 이 중 6핀은 PWM 출력 지원 |
| 아날로그 입력 | 6채널, 10비트 ADC |
| 통신 | UART, SPI, I²C / ATmega16U2가 USB와 직렬 통신 연결 |
| 전원과 논리 전압 | 동작 논리 5V / USB 또는 외부 전원 입력 |
| 보드 요소 | USB Type-B, 전원 잭, 리셋 버튼, ICSP 헤더 |

사진 중앙의 길쭉한 ATmega328P는 CPU뿐 아니라 메모리와 타이머 등도 포함한다. 가장자리 헤더는 센서와 확장 실드를 연결하는 통로다. 플래시는 프로그램 저장, SRAM은 변수와 스택, EEPROM은 전원 차단 후에도 남겨야 하는 설정값 저장에 이용할 수 있다. [2][3][4]

하드웨어 사양: [2] UNO R3 공식 소개, [3] 공식 데이터시트, [4] ATmega328P 자료.

## 03 Arduino UNO R3의 동작과 활용

### 명령어와 데이터의 저장 경로

ATmega328P는 프로그램과 데이터의 메모리 공간 및 접근 경로를 구분하는 AVR 계열 MCU이다. 프로그램 명령어는 플래시에 저장되고, 실행 과정에서 변하는 데이터는 주로 SRAM에 놓인다. 이는 강의에서 설명한 하버드 구조를 구체적인 하드웨어와 연결해 볼 수 있는 예이다. EEPROM까지 모두 “메모리”라고 부르지만 저장 대상과 접근 방법은 서로 다르다. [1][4]

### 입력에서 출력까지의 제어 과정

예를 들어 조도에 따라 LED 밝기를 바꾸는 장치를 설계할 수 있다. 조도 센서의 아날로그 전압을 ADC가 정수로 바꾸면, CPU는 그 값을 읽어 밝기 설정값을 계산한다. 이후 타이머의 PWM 설정을 바꾸면 출력 신호의 듀티비가 달라진다. 입력 변환, 데이터 처리, 출력 제어라는 컴퓨터의 기본 기능이 작은 보드 안에서 연속적으로 일어나는 것이다.

10비트 ADC의 코드는 0~1023이다. 기준 전압이 5V라고 가정하면 이상적인 한 단계의 크기는 약 5/1024V, 즉 4.88mV이다. 이는 분해능 계산이며 실제 정확도를 뜻하지는 않는다. 센서 오차, 기준 전압 변동, 잡음도 측정 결과에 영향을 준다. PWM 역시 핀에서 연속적인 아날로그 전압을 직접 만드는 기능과 구별해야 한다.

### 개발 방식과 자원 제약

일반적인 Arduino 프로그램은 초기화를 담당하는 setup과 반복 작업을 담당하는 loop로 구성된다. 개발 환경이 소스 코드를 기계어로 변환하고 보드에 기록하면, 리셋 후 저장된 프로그램이 실행된다. 프로그램을 기록하는 경로와 센서 데이터를 처리하는 경로를 구분하면 USB 연결과 CPU 실행의 관계도 이해하기 쉽다. [2][3]

설계에서 특히 주의할 자원은 2KB SRAM이다. 예를 들어 16비트 센서 값 500개를 배열로 보관하면 데이터만 1,000바이트가 필요하다. 이는 계산 예시이며, 남은 공간에는 다른 변수와 함수 호출에 필요한 스택까지 들어가야 한다. 따라서 긴 문자열이나 큰 배열을 무심코 사용하면 코드가 플래시에 들어가더라도 실행 중 문제가 생길 수 있다.

### 적합한 용도와 한계

UNO R3는 버튼·센서 입력, 간단한 모터 구동 신호, 소규모 자동제어의 구조를 학습하기에 적합하다. 반면 무선 통신은 별도 모듈이 필요하며, 큰 영상이나 복잡한 네트워크 처리를 맡기기에는 메모리와 연산 자원이 제한적이다. 모터는 GPIO에 직접 연결하는 대신 드라이버와 별도 전원 조건을 검토해야 한다. 처리 기능뿐 아니라 실제 부하를 구동할 전기적 조건까지 설계 범위에 포함된다.

구조·개발 근거: [1]~[4]. 조도 제어와 메모리 사용량은 사양을 적용한 설계 예시이다.

## 04 ESP32 DevKitC V4의 하드웨어 구성

ESP32-DevKitC V4는 무선 기능을 포함한 ESP32 모듈을 USB 전원과 핀 헤더로 쉽게 활용하도록 만든 개발보드이다. 여기서는 ESP32-WROOM-32E-N4를 기준으로 한다. ESP32-S3나 ESP32-C3는 다른 칩 계열이므로 본 표의 사양을 그대로 적용하지 않는다. [5][6]

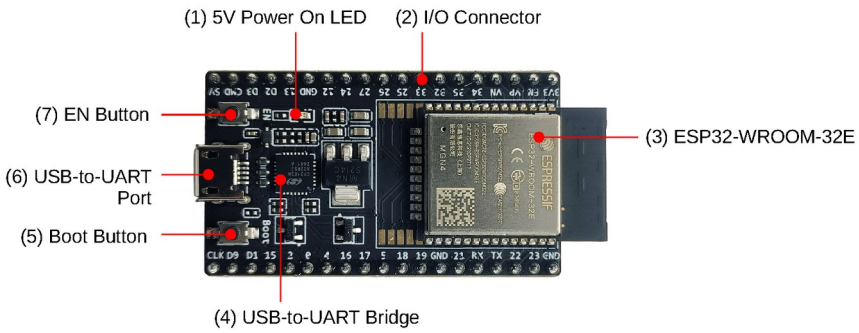



**그림 2. ESP32-DevKitC V4의 주요 부품. 출처: Espressif 공식 사용 안내 [5].**

| 구성 항목 | 조사 구성의 사양과 역할 |
|---|---|
| 프로세서 | Xtensa 32비트 LX6 듀얼코어, 최대 240MHz |
| 칩 내부 메모리 | SRAM 520KB, ROM 448KB |
| 모듈 저장 공간 | N4 구성: SPI 플래시 4MB, PSRAM 없음 |
| 무선 통신 | 2.4GHz Wi-Fi 802.11b/g/n, Bluetooth 4.2 및 BLE |
| 주변장치 | GPIO, ADC, DAC, PWM, UART, SPI, I²C 등 |
| 개발보드 부품 | Micro-USB, USB-UART 브리지, EN·Boot 버튼 |
| 전압 | GPIO 논리 3.3V / 보드의 USB 전원 입력과 구분 |

금속 차폐 덮개가 있는 부분은 ESP32 모듈이고, 그 끝의 PCB 안테나가 무선 신호를 송수신한다. USB-UART 브리지는 PC와 펌웨어 다운로드 통신을 연결한다. SRAM 520KB 전체가 응용 프로그램의 자유 공간은 아니며 실행 코드·시스템 구성에 따라 사용 가능량이 달라진다. [5][6][7]

사양: [5] 보드 사용 안내, [6] 모듈 데이터시트, [7] ESP32 데이터시트. 사진은 모듈 계열의 외관 예시이다.

## 05 ESP32 DevKitC V4의 통신과 제어

### MCU와 무선 통신의 결합

ESP32의 특징은 센서 제어와 네트워크 연결을 작은 구성에서 함께 구현할 수 있다는 점이다. 온습도 센서에서 읽은 값을 무선으로 서버에 전송하는 장치를 생각하면, 센서 인터페이스가 입력을 맡고 CPU가 데이터를 정리하며 Wi-Fi 기능이 외부 전송을 담당한다. 디지털 센서의 I²C 통신과 Wi-Fi의 무선 통신은 연결 대상과 전송 방식이 서로 다른 계층이다.

### 펌웨어와 실시간 운영체제

공식 개발 환경인 ESP-IDF에는 듀얼코어를 지원하는 FreeRTOS가 포함된다. 따라서 센서 읽기, 통신, 상태 표시 등을 태스크로 나누어 설계할 수 있다. 두 코어가 있다는 사실만으로 모든 작업이 자동으로 두 배 빨라지지는 않는다. 태스크의 우선순위, 코어 배치, 공유 메모리 접근과 동기화에 따라 실제 수행이 달라진다. [8]

예를 들어 센서 태스크는 일정 주기로 측정하고 전송 태스크는 큐에서 데이터를 꺼내 서버로 보낼 수 있다. 네트워크 연결이 잠시 끊어져도 측정이 계속되도록 역할을 나누는 것이다. 다만 큐의 크기를 정하고 데이터가 쌓일 때 버릴지 저장할지 정책을 정해야 한다. 이는 CPU의 병렬 실행뿐 아니라 제한된 메모리의 관리 문제이기도 하다.

### 플래시와 핀의 공유 자원

선정한 N4 모듈의 4MB 플래시는 비휘발성 저장 공간이다. 이 용량을 SRAM 520KB와 합쳐 하나의 RAM 용량으로 표시하면 안 된다. 프로그램·설정·업데이트 영역을 어떻게 나누는지에 따라 응용에 사용할 저장 공간도 달라진다. 같은 WROOM-32E 계열에도 플래시 용량과 PSRAM 유무가 다른 부품이 있으므로 정확한 부품 번호를 확인해야 한다. [6]

헤더의 모든 핀이 자유롭게 사용할 수 있는 GPIO인 것도 아니다. 보드 안내서는 D0·D1·D2·D3·CMD·CLK가 SPI 플래시 연결에 쓰인다고 설명한다. 이 핀들을 일반 입출력처럼 사용하면 코드 저장 공간 접근에 문제가 생길 수 있다. 핀의 수를 세는 것보다 핀의 실제 기능과 중복 사용 조건을 확인하는 것이 중요하다. [5]

### 적합한 용도와 전력 설계

무선 환경 모니터링, 스마트홈 센서, 간단한 원격 제어가 적합한 활용 예이다. 배터리 장치에서는 계속 통신하는 방식보다 측정·전송 후 절전하는 동작을 고려할 수 있다. 다만 칩의 절전 수치를 개발보드 전체의 소비 전류로 간주해서는 안 된다. 전원 LED, 레귤레이터, USB 브리지, 외부 센서도 전력을 소비하기 때문이다. 정확한 배터리 수명은 실제 보드와 동작 주기를 기준으로 측정해야 한다.

근거: [5]~[8]. 태스크 분리와 전력 검토는 실제 측정 결과가 아닌 설계 분석이다.

## 06 Raspberry Pi 5의 하드웨어 구성

Raspberry Pi 5는 운영체제를 실행하는 SBC이다. 본 보고서의 기준은 4GB 모델이며, 같은 Pi 5 제품군에도 RAM 용량이 다른 모델이 존재한다. MCU 보드보다 큰 메모리와 고속 입출력을 갖추어 영상·네트워크·사용자 인터페이스를 함께 처리하는 시스템에 활용할 수 있다. [9][10]

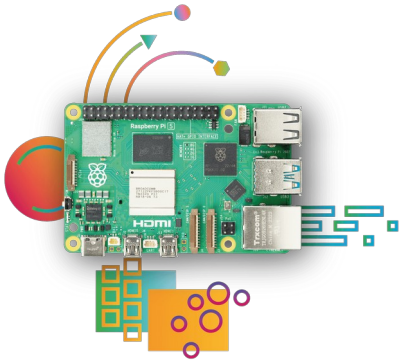


**그림 3. Raspberry Pi 5 보드. 출처: Raspberry Pi 공식 제품 페이지 [9].**

| 구성 항목 | 4GB 모델의 사양과 역할 |
|---|---|
| CPU | BCM2712, 64비트 Arm Cortex-A76 4코어, 2.4GHz |
| 캐시 | 코어당 L2 512KB, 공유 L3 2MB |
| 주기억장치 | 4GB LPDDR4X-4267 SDRAM |
| 영상과 입출력 | VideoCore VII GPU, RP1 입출력 컨트롤러 |
| 저장·확장 | microSD, PCIe 2.0 x1 / NVMe는 별도 어댑터 필요 |
| 외부 연결 | USB 3.0 2개, USB 2.0 2개, Gigabit Ethernet |
| 무선·디스플레이 | 듀얼밴드 Wi-Fi, Bluetooth 5.0, micro-HDMI 2개 |
| 전원 | USB-C, 5V/5A 전원 공급 권장 구성 |

40핀 헤더는 센서와 제어 신호를 연결하는 확장부이며, 40개 전체가 GPIO는 아니다. 보드의 RAM은 실행 중 작업 공간이고 microSD는 운영체제와 파일을 보관하는 저장장치다. 별도 SSD를 연결할 수 있어도 SSD가 CPU의 RAM을 대체하는 것은 아니다. [9][11]

사양 및 사진: [9] Raspberry Pi 5 제품 페이지, [10] Product Brief. 인터페이스: [11].

## 07 Raspberry Pi 5의 운영체제와 처리 구조

### 부팅과 프로그램 실행

Linux 기반 시스템에서는 부팅 과정을 거쳐 운영체제가 실행되고, 응용 프로그램이 메모리와 장치 서비스를 이용한다. 사용자가 저장한 프로그램 파일과 실행 중 프로세스의 메모리 상태는 서로 다르다. 파일은 microSD나 SSD에 보관되고, 실행에 필요한 코드와 데이터는 운영체제의 관리 아래 RAM에 적재된다. 이 구분은 강의의 주기억장치와 보조기억장치 개념을 실제 시스템에 적용한 것이다. [1][11]

### 멀티코어와 캐시 계층

Pi 5의 4개 CPU 코어는 병렬 작업을 처리할 여지를 제공한다. 그러나 작업을 나눌 수 있는지, 데이터에 함께 접근할 때 기다림이 생기는지, 저장장치가 충분히 빠른지에 따라 전체 처리 시간은 달라진다. 코어 수와 클럭은 중요한 사양이지만 그것만으로 응용 성능을 단정할 수는 없다.

코어당 L2 캐시와 공유 L3 캐시는 자주 쓰는 코드·데이터의 접근을 돕는 작은 고속 기억장치이다. CPU와 RAM 사이의 속도 차이를 완화하는 계층으로 이해할 수 있다. RAM 용량을 늘리는 것과 캐시 적중률을 높이는 것은 다른 문제이므로, 처리 지연이 생겼을 때는 메모리 부족인지 반복적인 데이터 접근 때문인지 구분해야 한다. [9]

### 영상 기반 출입 감지 장치의 예

카메라 입력을 받아 프레임을 메모리에 두고, 응용 프로그램이 사람의 존재를 판단한 뒤 화면 표시나 이벤트 전송을 수행하는 구조를 생각할 수 있다. 이때 입력 장치, RAM, CPU, 네트워크가 하나의 처리 경로를 이룬다. 원본 영상을 계속 저장하면 저장 공간과 쓰기 속도가, 여러 프레임을 동시에 처리하면 RAM과 CPU 자원이 주요 제약이 될 수 있다. 실제 처리 가능 프레임 수는 알고리즘과 입력 해상도를 정한 뒤 측정해야 한다.

### GPIO 제어와 운영상의 한계

Pi의 GPIO는 3.3V 신호를 기준으로 사용해야 한다. 또한 MCU처럼 아날로그 센서 전압을 읽는 범용 ADC가 기본 헤더에 제공되는 것은 아니므로, 아날로그 입력이 필요하면 외부 ADC 등을 검토해야 한다. 이는 “핀에 연결할 수 있다”는 사실과 “원하는 신호를 바로 처리할 수 있다”는 사실이 다름을 보여 준다. [11]

일반적인 Linux 사용자 프로그램은 운영체제의 스케줄링 영향을 받는다. 따라서 정밀한 펄스 발생을 반복문만으로 구현하면 시간 간격이 흔들릴 수 있다. 엄격한 제어 주기가 필요하면 하드웨어 주변장치나 별도 MCU에 역할을 나누는 설계가 적절하다. 지속적인 고부하에서는 냉각과 전원 공급도 함께 검토해야 한다. 공식 자료는 Pi 5용 냉각 장치와 충분한 전원 공급 구성을 안내한다. [9][10]

근거: [1][9]~[11]. 출입 감지 사례는 구성 이해를 위한 예시이며 실측 성능을 제시한 것이 아니다.

## 08 BeagleBone Black의 하드웨어 구성

BeagleBone Black Rev C는 TI Sitara AM3358을 탑재한 SBC이다. Linux를 실행하는 Arm 코어에 더해 별도의 PRU를 활용할 수 있다는 점이 특징이다. 일반 응용 기능과 시간 제약이 큰 입출력을 나누어 설계하는 관점에서 조사 가치가 있다. [12][14]

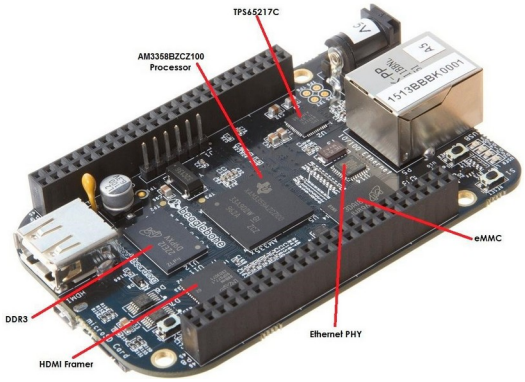



**그림 4. BeagleBone Black의 주요 부품 위치. 출처: BeagleBoard 문서 [12], Key Components. CC BY-SA 4.0.**

| 구성 항목 | Rev C 기준 사양과 역할 |
|---|---|
| 주 프로세서 | TI Sitara AM3358, 32비트 Arm Cortex-A8, 1GHz |
| 실시간 보조 처리 | PRU-ICSS, 200MHz PRU 2개 |
| 주기억장치 | 512MB DDR3L |
| 비휘발성 저장 | 4GB eMMC, 확장·부팅용 microSD 슬롯 |
| 외부 연결 | 10/100 Ethernet, USB 2.0 Host, mini-USB Client |
| 확장 헤더 | P8·P9, 각각 46핀 / GPIO·UART·SPI·I²C 등 |
| 신호 전압 | 디지털 I/O 3.3V, 아날로그 입력 최대 1.8V |

사진에서 프로세서, DDR 메모리, eMMC가 서로 다른 부품임을 확인할 수 있다. 이들은 각각 연산, 실행 중 작업 공간, 비휘발성 저장을 담당한다. 전원 관리 IC는 여러 부품에 필요한 전원 공급을 관리한다. 일반 Black 모델에는 Wi-Fi가 기본 탑재되어 있지 않으며 Wireless 모델과 구분해야 한다. [12][13]

사양: [12][13] BeagleBoard 하드웨어 문서, [14] TI AM3358 공식 자료.

## 09 BeagleBone Black의 실시간 입출력

### Arm 코어와 PRU의 역할 분담

PRU는 Programmable Real-Time Unit의 약자이다. AM3358의 PRU-ICSS는 Arm 코어와 별도로 동작할 수 있는 서브시스템으로, 빠른 입출력 반응이나 사용자 정의 인터페이스 구현에 활용된다. Linux에서 파일·통신·사용자 인터페이스를 처리하면서 PRU에 시간에 민감한 작업을 맡기는 형태를 설계할 수 있다. [14]

이 구조를 1GHz CPU와 200MHz PRU 중 어느 쪽이 더 빠른지로만 비교하면 목적을 놓치게 된다. 주 CPU는 복잡한 소프트웨어를 실행하고, PRU는 정해진 입출력 흐름에 집중하도록 사용할 수 있다. 서로 다른 처리기에 적절한 역할을 주는 것이 핵심이며, 주파수를 더해 하나의 성능 수치로 표시할 수는 없다.

### 회전 센서 입력을 수집하는 장치의 예

회전 센서에서 짧은 간격으로 발생하는 펄스를 수집하고, 사용자가 화면에서 회전 상태를 확인하는 장치를 생각할 수 있다. 입력 펄스 처리와 시간 정보 수집을 PRU에 맡기고, Arm 측 프로그램은 일정 단위로 전달받은 결과를 저장하거나 네트워크로 보낼 수 있다. 운영체제의 다른 작업이 많아져도 입력 수집을 별도 처리기로 분리할 수 있다는 것이 이 설계의 목적이다.

그렇다고 PRU를 사용하는 모든 응용의 실시간성이 자동 보장되는 것은 아니다. PRU 코드의 실행 경로, 메모리 접근, 코어 사이의 데이터 전달 방식, 버퍼가 가득 찼을 때의 동작까지 검토해야 한다. 평균 처리 시간뿐 아니라 최악의 지연과 누락 여부를 측정해야 정확한 제어 요구를 만족하는지 판단할 수 있다.

### 핀 다중화와 확장 보드

P8·P9 헤더는 풍부한 신호 연결을 제공하지만 모든 기능을 동시에 사용할 수 있는 것은 아니다. 일부 신호는 eMMC나 HDMI 등 보드의 기존 기능과 공유된다. 따라서 확장 보드인 Cape를 연결하거나 GPIO 기능을 바꿀 때는 현재 핀 설정과 충돌 여부를 확인해야 한다. 헤더 핀 수는 실제로 독립 사용 가능한 GPIO 수와 다르다. [12][13]

특히 아날로그 입력의 최대 전압은 1.8V이므로 3.3V 디지털 신호 규격과 혼동하면 안 된다. 외부 센서가 더 높은 전압을 내보낸다면 적절한 신호 조정 회로가 필요하다. 프로세서의 기능 목록만 읽는 것보다 실제 보드의 연결과 전기적 한계를 함께 읽어야 한다는 점을 보여 준다. [12]

### 적합한 용도와 한계

BeagleBone Black은 Linux 응용과 정밀 입출력을 결합하는 제어 학습, 데이터 수집, 사용자 정의 인터페이스 실험에 적합하다. 반면 512MB RAM과 기존 세대의 CPU는 큰 영상 처리나 여러 무거운 응용의 동시 실행에 제약이 된다. PRU 개발과 핀 설정에는 추가 학습이 필요하므로, 단순 LED 제어만 한다면 더 작은 MCU 보드가 구현을 단순하게 만들 수 있다.

근거: [12]~[14]. PRU 활용과 역할 분담은 자료를 바탕으로 구성한 설계 예시이다.

## 10 보드 비교와 목적에 따른 선택

| 비교 기준 | UNO R3 | ESP32 V4 | Pi 5 4GB | BeagleBone Black |
|---|---|---|---|---|
| 중심 구조 | 8비트 MCU | 32비트 MCU | 64비트 SBC | 32비트 SBC |
| CPU | 16MHz<br>1코어 | 최대 240MHz<br>2코어 | 2.4GHz<br>4코어 | 1GHz<br>1코어와 PRU |
| RAM | 2KB | 520KB[1] | 4GB | 512MB |
| 저장 공간 | 플래시 32KB<br>EEPROM 1KB | 플래시 4MB[2] | microSD 등<br>별도 준비 | eMMC 4GB<br>microSD 확장 |
| 무선 내장 | 없음 | Wi-Fi·BT | Wi-Fi·BT | 없음 |
| 대표 실행 환경 | 제어 펌웨어 | ESP-IDF<br>FreeRTOS | Linux | Linux와<br>PRU 펌웨어 |
| 핵심 강점 | 간단한 제어<br>구조 학습 | 무선 센서<br>통신 결합 | 영상·서버<br>복합 응용 | 입출력 시간<br>제약 대응 |

표 1. 조사 모델 비교. 근거: [2]~[14]. [1] 칩 내부 SRAM 명목 용량. [2] WROOM-32E-N4 기준. 최대 클럭과 메모리 명목 용량은 실측 응용 성능이 아니다.

### 요구사항을 먼저 정하는 선택 절차

첫째, 입력과 출력의 종류를 정한다. 센서가 아날로그인지 디지털인지, 모터 구동 회로가 필요한지, 외부 통신이 필요한지에 따라 주변장치 구성이 달라진다. 둘째, 실행 중 데이터 양과 저장할 데이터 양을 따로 계산한다. 셋째, 응답 시간의 허용 범위와 전원 조건을 정한다. 마지막으로 개발 환경과 유지보수 부담을 비교한다.

### 동일한 온실 관리 과제에 적용

온도에 따라 팬을 켜는 기능만 필요하다면 UNO R3를 검토할 수 있다. 측정값을 무선으로 전송해야 한다면 ESP32가 구성을 줄일 수 있다. 카메라 영상과 현장 화면까지 통합한다면 Pi 5가 적합한 후보가 된다. Linux 기반 관리와 정밀한 펄스 입출력을 함께 구현해야 한다면 BeagleBone Black의 PRU 활용을 검토할 수 있다. 이는 요구사항에 따른 선택 예시이며 모든 상황의 우열 순위는 아니다.

### 컴퓨터 구조 관점의 종합

네 보드 모두 CPU·기억장치·입출력·연결 통로를 갖추지만, 자원을 묶는 방식과 소프트웨어 계층이 다르다. MCU의 작은 메모리, SBC의 캐시와 외부 DRAM, 무선 통신부, PRU 같은 전용 처리기는 각자의 목적에 맞춘 구조적 선택이다. 임베디드 시스템의 좋은 설계는 가장 큰 사양을 고르는 것이 아니라 요구 기능을 안정적으로 수행할 구조를 고르는 일이라고 판단한다.

## 11 참고문헌과 그림 출처

**자료 확인일: 2026년 9월 24일. 본문의 [번호]는 아래 자료를 가리킨다. 제조사·공식 재단의 자료를 우선 사용했으며, 그림 출처는 각 그림 설명에도 표기하였다.**

**[1] 강의 자료**
오세춘, 「제1장 컴퓨터시스템 개요」, 제공된 PDF. PDF 6~8·11~13·56·62쪽. 과제 작성 조건은 「[개인과제-2] 임베디드 시스템 개발용 보드 조사」 PDF 1쪽 참조.

**[2] Arduino, UNO R3**
https://docs.arduino.cc/hardware/uno-rev3/

**[3] Arduino, UNO R3 User Manual · 그림 1**
https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf

**[4] Microchip, ATmega328P**
https://www.microchip.com/en-us/product/atmega328p

**[5] Espressif, ESP32-DevKitC V4 User Guide · 그림 2**
https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user_guide.html

**[6] Espressif, ESP32-WROOM-32E / 32UE Datasheet**
https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32e_esp32-wroom-32ue_datasheet_en.pdf

**[7] Espressif, ESP32 Series Datasheet**
https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

**[8] Espressif, ESP-IDF Programming Guide, FreeRTOS**
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html

**[9] Raspberry Pi, Raspberry Pi 5 · 그림 3**
https://www.raspberrypi.com/products/raspberry-pi-5/

**[10] Raspberry Pi, Raspberry Pi 5 Product Brief**
https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf

**[11] Raspberry Pi, Computer Hardware Documentation**
https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

**[12] BeagleBoard, BeagleBone Black Overview · 그림 4**
https://docs.beagleboard.org/boards/beaglebone/black/ch04.html

**[13] BeagleBoard, BeagleBone Black High Level Specification**
https://docs.beagleboard.org/boards/beaglebone/black/ch05.html

**[14] Texas Instruments, AM3358**
https://www.ti.com/product/AM3358

## 12 개인 소감

### 성능을 바라보는 기준

이번 조사에서 가장 의미 있다고 느낀 점은 개발보드의 성능을 하나의 숫자로 판단하기 어렵다는 것이다. 클럭과 메모리 용량을 나열하면 Raspberry Pi 5가 가장 넉넉한 시스템처럼 보인다. 그러나 센서 값을 읽어 간단한 출력을 내는 장치라면 그만큼의 운영체제와 연산 자원이 꼭 필요한 것은 아니다. 주어진 기능을 어떤 조건에서 수행할 것인지가 보드 선택보다 먼저 정해져야 한다고 생각한다.

### 강의 내용과 실제 부품의 연결

컴퓨터 구조에서 배운 CPU, 기억장치, 입출력장치, 시스템 버스가 추상적인 분류에 그치지 않는다는 점도 확인할 수 있었다. UNO R3에서는 한 MCU 안의 플래시와 SRAM이 서로 다른 역할을 맡고, Raspberry Pi와 BeagleBone에서는 프로세서 옆의 RAM과 저장장치가 분리되어 있다. 같은 “메모리 용량”이라는 표현도 실제로는 서로 다른 자원을 뜻할 수 있다는 점이 특히 중요하게 느껴졌다.

ESP32를 조사하면서는 프로세서, 모듈, 개발보드의 차이를 구분해야 한다는 점을 배웠다. ESP32라는 이름만으로 모든 보드의 플래시 용량과 핀 기능을 설명할 수는 없다. 부품 번호와 공식 문서를 함께 확인하는 과정이 필요하다. 소프트웨어를 작성할 때도 실행 환경을 정확히 알아야 하듯, 하드웨어 역시 제품 이름보다 실제 구성을 확인하는 습관이 필요하다고 생각한다.

### 실시간 처리에 대한 이해

BeagleBone Black의 PRU는 빠른 CPU와 정해진 시간에 반응하는 시스템이 같은 의미가 아니라는 점을 보여 주었다. 복잡한 작업을 처리하는 Linux와 시간에 민감한 입출력을 분담하는 방식이 인상적이었다. 임베디드 시스템에서는 결과가 맞는지뿐 아니라 언제 결과가 나오는지도 설계의 일부라는 점을 더 명확하게 이해하게 되었다.

### 앞으로 확인하고 싶은 내용

컴퓨터공학 전공 4학년으로서, 앞으로는 라이브러리 사용법에만 집중하기보다 데이터가 센서에서 CPU와 메모리를 거쳐 출력으로 이동하는 과정을 함께 살펴보고 싶다. 직접 실습한다면 먼저 ESP32로 센서 값을 읽고 무선으로 전송하는 작은 시스템을 만든 뒤, 통신 지연과 메모리 사용량을 기록해 보고 싶다. 이후 영상 기능이 필요할 때 Pi를 연결하는 방식으로 기능을 나누어 보면 이번에 조사한 구조의 차이를 더 구체적으로 확인할 수 있을 것 같다.

이번 보고서는 공식 문헌을 바탕으로 한 조사이므로 실제 보드에서의 소비 전력이나 최악의 응답 시간을 확인한 것은 아니다. 다음 단계에서는 같은 요구사항을 여러 보드로 구현하고, 구현 난이도와 동작 지연을 직접 비교해 보고 싶다. 사양표를 읽는 능력과 실제 동작을 검증하는 능력을 함께 기르는 것이 이번 조사에서 얻은 가장 중요한 학습 방향이다.